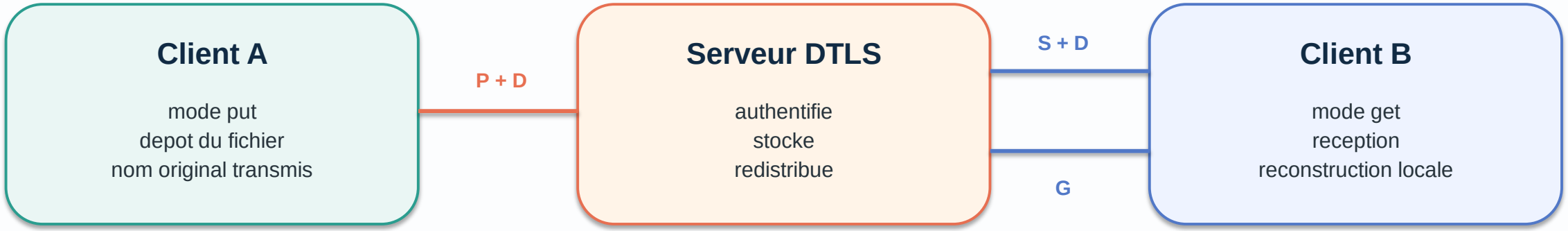
Transfert UDP/DTLS securise

DTLS-PSK avec AES-256-GCM, controle d'acces et stockage multi-fichiers

Le fichier est chiffre avant le transport UDP, verifie a la reception, puis dechiffre uniquement si son integrite est valide.

Architecture generale

Le serveur agit comme relais securise entre les clients



Le transfert n'est pas un envoi direct A vers B.

A depose le fichier sur un serveur connu par IP et port.

B recupere le dernier fichier disponible sur ce meme serveur.

Toutes les donnees utiles passent dans une session DTLS chiffree.

Securite imposee dans le code

Une seule suite cryptographique est acceptee

Suite unique: PSK-AES256-GCM-SHA384.

AES-128 n'est pas propose et n'est pas accepte.

Le handshake echoue si un cote ne supporte pas cette suite exacte.

La suite negociee est affichee dans les journaux du client et du serveur.

```
SSL_CTX_set_cipher_list(ctx,
    "PSK-AES256-GCM-SHA384");
```

Element	Role
PSK	Authentification par secret partage.
AES-256	Chiffrement symetrique avec une cle de 256 bits.
GCM	Chiffrement authentifie + tag d'integrite.
SHA-384	Hachage utilise par la suite DTLS pendant le handshake.

Authentification DTLS-PSK

Le serveur accepte seulement les identites declarees

Le client presente son identite PSK pendant le handshake.

Le serveur cherche cette identite dans sa configuration.

Si le secret ne correspond pas, la session DTLS ne s'etablit pas.

Apres authentification, le serveur applique les droits put et get.

Secrets et utilisateurs

Les secrets sont externalises dans des fichiers de configuration

```
psk_client.txt
identity=Client1
key=<secret local>
```

```
psk_clients.txt
Client1=<secret local>,put,get
```

La cle PSK n'est plus une constante dans le code C++.

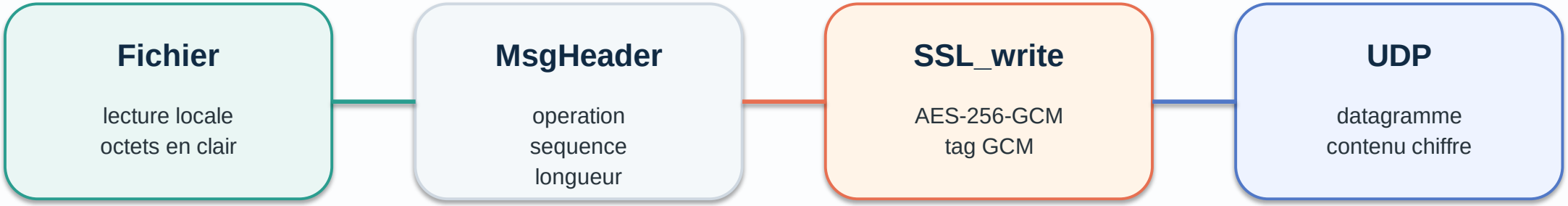
Changer une identite ou une cle ne demande pas de recompilation.

Le serveur peut definir plusieurs utilisateurs.

Les permissions sont associees a chaque identite.

Chiffrement pendant l'envoi

SSL_write transforme les messages applicatifs en enregistrements DTLS chiffres



Le fichier est decoupe en blocs de 8192 octets maximum.

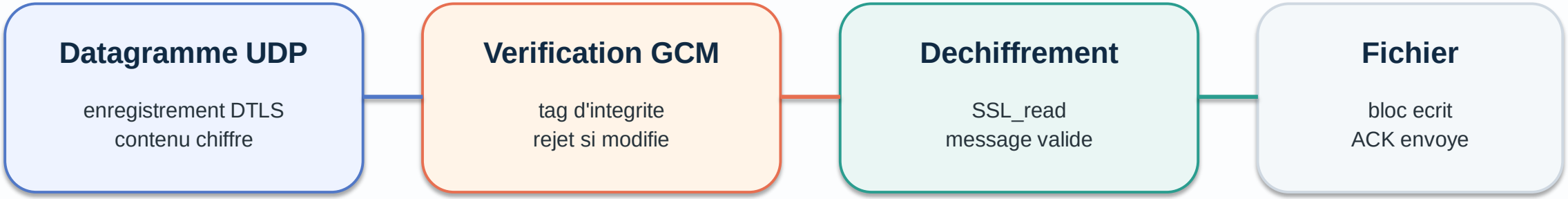
Chaque bloc est encapsule avec un en-tete applicatif.

OpenSSL chiffre le message complet dans la session DTLS.

Le reseau ne voit pas le bloc de fichier en clair.

Verification et dechiffrement

SSL_read remet les donnees au code seulement si l'integrite est valide



Si un paquet est modifie, GCM detecte l'anomalie.

Un message invalide n'est pas remis au protocole applicatif.

Un message valide est dechiffre, puis son numero de sequence est verifie.

Le bloc attendu est ecrit dans le fichier final.

Protocole applicatif

Les messages P/G/S/D/A/E organisent le transfert du fichier

Code	Sens	Role
P	Client -> Serveur	Debut d'un depot: taille totale + nom original.
G	Client -> Serveur	Demande du dernier fichier stocke.
S	Serveur -> Client	Taille totale + nom original du fichier.
D	Deux sens	Bloc de donnees numerote.
A	Deux sens	Accuse de reception.
E	Serveur -> Client	Erreur ou acces refuse.

Le protocole applicatif circule lui aussi dans DTLS.

Les numeros de sequence permettent de reconstruire le fichier dans l'ordre.

Les ACK compensent les pertes possibles de UDP.

Depot d'un fichier

Mode put: le serveur stocke un nouveau fichier sans ecraser les anciens

- Le client lit le fichier et calcule sa taille totale.
- Le message P annonce la taille et le nom original.
- Le fichier est envoye par blocs D numerotes.
- Le serveur confirme chaque bloc avec A.
- Le fichier est enregistre dans stored_files avec horodatage et proprietaire.

```
stored_files\20260416_115131_Client1_rapport_origina
l.txt
stored_files\latest.txt
```

Stockage

plusieurs depots conserves
dernier fichier reference
nom original nettoye

Recuperation d'un fichier

Mode get: le client reconstruit le fichier avec son nom original

- Le client envoie G au serveur.
- Le serveur verifie le droit get de l'identite PSK.
- Le serveur lit latest.txt pour retrouver le dernier fichier.
- Le message S renvoie la taille et le nom original.
- Le client ecrit les blocs dans le dossier choisi avec l'extension originale.

Exemple

rapport_original.txt
nom conserve
extension conservee

Validation

hash SHA-256 identique
transfert termine
AES-256-GCM confirme

Fiabilite au-dessus de UDP

UDP est rapide, mais le protocole ajoute ACK et retransmissions

- UDP ne garantit pas la livraison de chaque datagramme.
- Chaque bloc D possede un numero de sequence.
- Le recepteur renvoie A pour confirmer le bloc recu.
- Sans ACK, l'emetteur renvoie le meme bloc jusqu'a une limite d'essais.
- Les doublons sont confirmes mais ne sont pas reecrits dans le fichier.

Securite

DTLS
AES-256-GCM
integrite cryptographique

Fiabilite

ACK
sequence
retransmission

Controle d'acces

Les operations sont autorisees selon l'identite PSK

Operation	Droit requis	Effet si absent
Depot	put	Le serveur refuse de stocker le fichier.
Recuperation	get	Le serveur refuse de redistribuer le fichier.
Handshake	identite + secret valides	La session DTLS echoue.

```
psk_clients.txt
Client1=<secret local>,put,get
```

Validation technique

Resultats observes pendant le test de transfert

```
Identite PSK client: Client1
Suite DTLS negociee: PSK-AES256-GCM-SHA384
Nom original envoye: rapport_original.txt
Nom original recu: rapport_original.txt
Transfert termine.
```

Synthese

Transport securise, acces controle et conservation des fichiers

UDP assure le transport des datagrammes.

DTLS-PSK authentifie les pairs et etablit une session securisee.

AES-256-GCM chiffre les donnees et detecte les modifications.

Le protocole P/G/S/D/A/E organise depot, recuperation, blocs et ACK.

Les fichiers gardent leur nom original et sont conserves dans un stockage multi-fichiers.

Les droits put/get limitent les operations autorisees par identite.

Le systeme transmet un fichier sur UDP tout en protegeant son contenu, son integrite et les operations autorisees.